

★ Your membership will expire on March 16, 2025 [Reactivate membership](#)

★ Member-only story

Gradient boosting— the most comprehensive comparison of Python libraries



Tina Wenzel · [Follow](#)

8 min read · Jan 23, 2024

Listen

Share

More



Gradient Boosted Machines — A residuals' worst enemy

Gradient boosting is one of the most used machine learning algorithms in data science. *Residual-based boosting* thereby leads the lot in predictive accuracy and adaptability, making it a usual suspect among Kaggle winning solutions and also widely used in industry for problems across ranking, classification and regression tasks. Since the first implementation in scikit-learn (2012) and XGBoost (2014), a number of new Python libraries have emerged: h2o (2015), LightGBM (2016) and CatBoost (2017). This leaves data enthusiasts today spoilt for choosing between the different libraries.

In this post, I outline how implementations differ and compare them along with general criteria and feature support. If you are considering gradient-boosted decision trees (GBDT), I hope this will help in understanding the capabilities and the trade-offs associated with a particular library to determine the best one for your use case.

In a nutshell,

residual-based boosting works by combining predictions from many models (ensemble of weak learners), each focusing on the weaknesses of the last model by fitting its residuals. The weighted combination of predictions thereby sequentially learns the more difficult patterns in the data and builds an increasingly accurate model that predicts “the hell out of your data”.

1) Scope of the libraries

One of the obvious differences across the implementations is the scope of the libraries. XGBoost, LightGBM and CatBoost are boosting-specific implementations, whereas scikit-learn and h2o are universal modelling libraries that cover a much larger set of models.

2) Supported boosting types

Another key difference is the boosting types they support. In general, boosting types differ in the way they control over-fitting (e.g. regularisation) and how they implement stochastic gradient descent (e.g. sub-sampling).

The Gradient Boosted Trees implementation in scikit-learn is the most generic implementation of GBDTs and based on Friedman (1, 2), who laid the groundwork

for a new generation of boosting algorithms. XGBoost is a more optimised version of the latter which builds trees in parallel, rather than sequential as in scikit-learn. It also has more fine-grained options for regularisation and over-fitting controls of the tree, e.g. decision trees with dropout (DART), which aims to prevent over-fitting by using a random subset of trees to calculate the residuals that the next tree will fit.

scikit-learn	XGBoost	LightGBM	CatBoost	h2o
Gradient Boosted Trees (GBDT) implementation based on Friedman using optimised CART	1) GBDT (by default) 2) Generalized Linear Model (GLM) 3) Dropout Additive Regression Trees (DART)	1) GBDT 2) Random Forest 3) DART 4) Gradient-based one-side sampling (GOSS)	1) Ordered/ Dynamic 2) Plain (faster; classic GBDT)	1) H2OGradientBoostingEst. builds generic GBDT 2) H2OXGBoostEst. (GBDT , DART, GLM)

Figure 1: Comparing supported boosting types across libraries

LightGBM and CatBoost are also optimised versions of generic GBDTs. In addition to generic GBDT and DART, LightGBM offers support for gradient-based one-side sampling (GOSS), which gives higher sampling weights to data points with larger gradients (slope of the tangent of the loss function) before each iteration. This speeds up the training process by excluding a large portion of small gradient points.

CatBoost has two boosting schemes — *ordered* and *plain*. The latter is the generic gradient boosted trees implementation. The default is *ordered*, which aims to avoid biased gradients by excluding the current observations for the model that calculates the current gradients. (See their latest paper for details).

h2o on the other hand currently has two ways of running gradient boosting. H2OGradientBoostingEstimator follows the generic GBDT algorithm specified in the Elements of Statistical Learning. Further, an implementation of XGBoost was recently added that supports DART, GLM and generic GBDT (See details on their blog and documentation).

3) Splitting strategies

Searching for the best split is the key computational burden in tree-learning. Since there is no analytical solution to determine which feature to split on next, the algorithm has to search through all available features and choose the best at each stage. A lot of innovation has come from this area recently and resulted in some key differences in the way a tree is grown in an implementation. Whereas scikit-learn and CatBoost build large symmetric trees, LightGBM and XGBoost tend to build more deep asymmetric trees. (h2o supports both.)

3.1) Growing policies: wide vs. best

If you were to draw a tree with a root node and 6 leafs, how would you build it?

Expand in pre-determined order

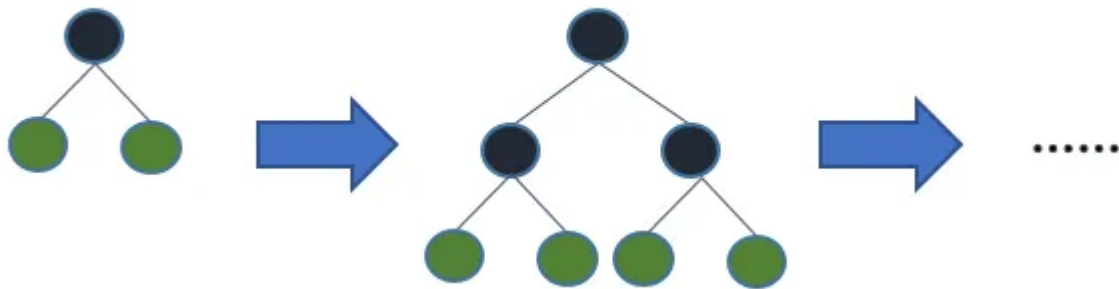


Figure 2: Depth-wise/ Level-wise expansion

If you expand it **level by level**, you grow the tree in a fixed order, i.e. expanding nodes from **left to right**, one level at a time. This is called “**depth-first**” expansion since we expand the tree along the same level. Traditional versions of decision tree algorithms such as C4.5 or CART (Classification and Regression Trees) follow a depth-first strategy. An advantage of it is that a balanced tree serves as regularisation and thus works well in smaller data where it prevents overfitting. CatBoost uses a special type of depth-first expansion called ***oblivious trees***. The key idea is that it uses a vectorised representation of the tree (binarised splitting criteria within each level), which can be evaluated faster.

Expand wherever it's best

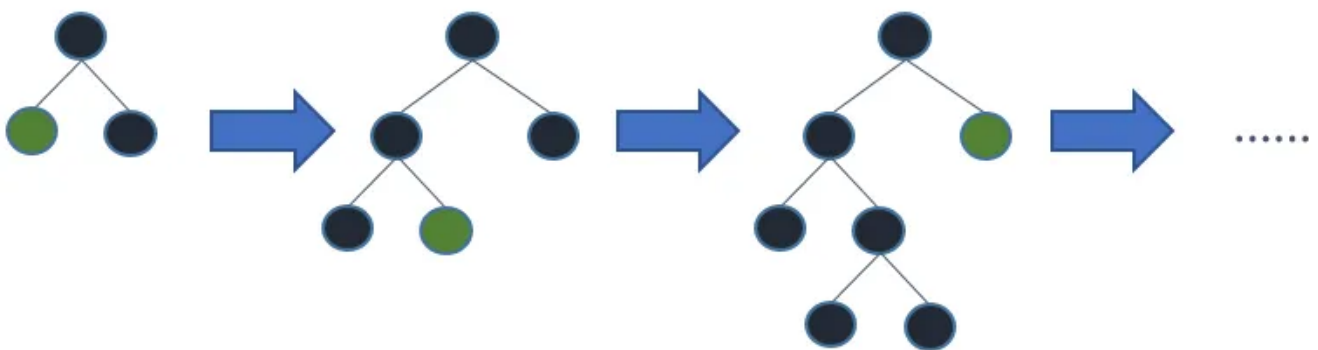


Figure 3: Best-first/ Leaf-wise expansion

Another expansion method is to expand whatever **leaf** is best (marked green in Figure 3). This is called “**best-first**” learning since the tree is expanded at the node with the largest reduction in the tree building criterion (e.g. Gini). Best-first decision

trees were introduced by Friedman et. al. (2000) and any traditional decision trees can be adapted to this growing policy.

Why is best-first faster?

Since best-first expands nodes based on their contribution to the global loss (Figure 3) and not just locally (Figure 2) on a particular level, nodes with less potential are left unexpanded. By stopping growth strategically, we are thus more likely to focus on the important nodes faster and thereby build a tree with better information gain, same amount of leaves but at lower computational cost.

Importance for bigger data

This really starts paying off for >10k rows, where best-first tends to converge much faster. On smaller datasets, best-first tends to overfit, which can be contained by limiting the *max_depth* parameter.

How to use it

When working with Python libraries, *growth_policy* is the parameter to tune in practice. Best-first learning is now supported in XGBoost, LightGBM, and h2o. It isn't supported yet in CatBoost and scikit-learn.

3.2) Splitting strategies: exact vs. approximate

Decision tree algorithms work by iteratively separating training data into smaller and smaller subsets. The decision rule to optimise the training objective is thereby searching for the best column to split and where to split it best. For high-cardinality features (e.g. continuous), iterating through *all* possible splitting points ("*exact*" evaluation) leads to significant computational cost.

Approximate methods, such as histogram-based binning avoid searching all possible splits by discretising continuous values into discrete bins. This reduces the computational complexity of exact splitting significantly since searching over bins is both faster and more memory-efficient.

How to use it

LightGBM was the first to include histogram-based splitting as default. XGBoost, h2o and CatBoost support it optionally. Scikit-learn does not support approximate splitting.

4) Scalability

When libraries are compared, speed and accuracy are often the first focus. Three interesting benchmarks have been published recently, which I would like to

mention here. Unfortunately, not all libraries will be covered equally and I cannot guarantee that boosting schemes covered were comparable.

— User benchmarks

In an application for price prediction, [Connor McNamara](#) compared GBM-specific libraries (LightGBM, CatBoost, and XGBoost).

— Findings

“In terms of model accuracy, XGBoost was the clear winner in both GridSearchCV and RandomizedSearchCV, with the lowest RMSE. For early stopping, LightGBM was the winner, with a slightly lower RMSE than XGBoost.

Overall, CatBoost was the clear underperformer, with training times comparable to XGBoost while having the worst predictions in terms of RMSE.”

Implementation	GridSearch CV time	GridSearch CV RMSE	Randomized SearchCV time	Randomized SearchCV RMSE	Early stopping time	Early stopping RMSE
xgboost	2064 s	478.03	1488 s	485.39	205 s	480.588
lightgbm	240 s	774.08	102 s	643.38	74 s	475.219
catboost	1908 s	2993.34	540 s	731.29	453 s	582.82

Figure 4: Reproduced from Connor McNamara

Some other interesting benchmarks have been done on the airline dataset (100m rows). [Szilar](#) benchmarked XGBoost against LightGBM and h2o on a 10k row sample of the data.

For CPU implementations, he found h2o had the best performance (followed by XGBoost) in terms of AUC, while LightGBM was fastest (2x) at a similar AUC performance. For GPU implementations, h2o crashed and XGBoost was both the fastest and most accurate in terms of AUC.

Interestingly, the airlines dataset was also [benchmarked](#) against XGBoost by Microsoft last year, where LightGBM came out fastest and also marginally better in terms of AUC.

— XGBoost.ai benchmark

In a recent blog post by [xgboost.ai](#), [CPU and GPU implementations](#) of the XGBoost histogram version, LightGBM and CatBoost were compared on different [datasets](#), including the above-mentioned airline dataset.

— Findings

	YearPrediction		Synthetic		Higgs		Cover Type	Bosch		Airline		
	Time(s)	RMSE	Time(s)	RMSE	Time(s)	Accuracy	Time(s)	Accuracy	Time(s)	Accuracy	Time(s)	Accuracy
xgb-cpu-hist	216.71	8.8794	580.72	13.6105	509.29	74.74	3532.26	89.2	810.36	99.45	1948.26	74.94
xgb-gpu-hist	30.39	8.8799	43.14	13.4606	38.41	74.75	107.70	89.34	27.97	99.44	110.29	74.95
lightgbm-cpu	30.82	8.8777	463.79	13.585	330.25	74.74	186.27	89.28	162.29	99.44	916.04	75.05
lightgbm-gpu	25.39	8.8777	576.67	13.585	725.91	74.7	383.03	89.26	409.93	99.44	614.74	74.99
cat-cpu	39.93	8.9933	426.31	9.387	393.21	74.06	306.17	85.14	255.72	99.44	2949.04	72.66
cat-gpu	10.15	9.0637	36.66	9.3805	30.37	74.08	N/A	N/A	N/A	N/A	303.36	72.77

Figure 5: Reproduced from XGBoost.ai blog post

The result showed XGBoost and LightGBM being very similar in terms of accuracy, both for CPU and GPU implementations (Unfortunately, F1 score or AUC was not provided). In terms of speed, LightGBM was fastest for CPU implementations, while XGBoost had the fastest GPU implementation, followed by CatBoost.

5) General criteria

In practice, speed and accuracy may not be the determining factors when choosing a certain tool over another. For example, the cost of false positives in industry applications is often high and accuracy will be more important than speed. Similarly, more interpretable, simple models which are less accurate may be preferable to complex models that are hard to interpret.

One might also look at how consistent the library is from version to version. In a previous [post](#) on model consistency, for example, I found that there can be considerable variation in results when running the same model but under a different version of the same library. This is particularly the case for CatBoost and

LightGBM, which had the best model performance but were also most sensitive to changes such as introducing NaN values and smaller datasets.

Last but not least, companies choose what fits into their toolchain the quickest, whichever is documented best or actively maintained. These and other general criteria are summarised in the below table.

General criteria	scikit-learn	XGBoost	LightGBM	CatBoost	h2o
scikit-learn API integration (This means you can use it like any other sklearn model)	N/A	Yes	Yes	Yes	No everything is different
scope	Universal model library and training framework	GBM and RF-specific	GBM and RF-specific	GBM and RF-specific	Universal model library and training framework
focus	universal library for machine learning	accuracy and speed	speed and accuracy	categorical features, auto-pre-processing and auto-tuning	auto- and distributed ML in production
written in	Python, Cython, C	C++	C++	C++	Java
model export	Yes e.g. JPMML-SkLearn	Yes e.g. JPMML-XGBoost, ONNX	Yes e.g. ONNX	Yes as Python/ C++ code/ CoreML	Yes MOJO/POJO
Spark version	spark-sklearn	Yes	Yes	under development; some support, see spark.py	Yes
GPU	No	Yes	Yes	Yes	Yes
multi-threading	No	Yes	Yes	Yes	platform-dependent: Yes, on Linux
parallel learning	Not internally; possible via dkllearn, spark-sklearn or Spark core	distributed xgboost on AWS; sparkxgboost, dask, Spark core	parallel_learning; mmlspark	See spark.py	Yes
Windows	Yes	Difficult	Yes	Yes	Yes
Linux	Yes	Yes	Yes	Yes	Yes
MacOS	Yes	Yes	Yes	Yes	Yes
version consistency	good	good	big variation version to version	good	good
commercial	No	No	No	No	partly driverless AI
documentation	great	great	good	good	good
release history	great	great	bad stopped in July 2017	great	none? occasional blog-posts with new features?

Figure 6: General comparison across libraries

The second table below compares the libraries from a more feature-specific angle.

Supports	scikit-learn	XGBoost	LightGBM	CatBoost	h2o
level-wise split	Yes	Yes	No	Yes	Yes
best-first split	No	Yes param: growth_policy	Yes	No	Yes param: growth_policy
histogram-based split	No	Yes param: tree_method	Yes	Yes	Yes param: histogram_type
over-fitting controls	Yes e.g. max_depth, regularisation	Yes e.g. max_depth, early stopping, min_child_weight, gamma	Yes e.g. early stopping, max_bin, regularisation	Yes e.g. balanced trees, regularisation, IncToDec, iter	Yes e.g. early stopping, regularisation, Driverless AI
categorical features	No pre-processing required	No pre-processing required	Yes param: cat_feature	Yes param: cat_feature	Yes param: nbins_cats, categorical_encoding
auto-encoding of categorical features	No	No	Yes 1 method, more under development	Yes 4 methods	Yes 9 methods
missing value handle	No pre-processing required	Yes default treatment	Yes customisable	Yes	Yes
auto-tuning	No	No	No	Some	Yes driverless AI
imbalanced data	No via pre-processing or eval metrics	Yes scale_pos_weight	Yes init_score, is_unbalance	Yes classification only: scale_pos_weight	Yes balance_classes
feature importance	Yes feature_importances_	Yes plot_importance, feature_importances_	Yes plot_importance	Yes, get_feature_importance, summary_plot	Yes but tedious
evaluation report	Yes e.g. confusion_matrix	Yes e.g. confusion_matrix	Yes e.g. confusion_matrix	Yes e.g. confusion_matrix	Yes model_performance, confusion_matrix
objective: ranking	Yes via multi-class classifier w. ranking loss	Yes Classifier w. objective rank: pairwise	Yes LGBMRanker or LGBMClassifier multiclass	Yes Classifier w. objective: YetiRank	Yes via multi-class classifier
objective: classification	Yes 2 losses	Yes 8 losses	Yes 9 losses	Yes 9 losses	Yes specified via dis-

[Open in app ↗](#)
Medium
 Search

 5


Endnotes

Open source gradient boosting libraries are evolving at a rapid pace. In this post, I compared the most used libraries for GBM in terms of some key implementation differences, feature support and performance benchmarks. Going beyond speed and accuracy, I also included a comparison of general criteria comparison that should help you think about some of these in your context.

Ultimately, once you're clear of the objective and constraints, nothing goes over doing your own benchmark. Since CatBoost, LightGBM and XGBoost all have scikit-

learn wrappers, you can do so very quickly (see example of this [here](#)).

My only goal is to gradient boost over myself of yesterday. And to repeat this every day with an unconquerable spirit. ([Mike Kim](#))

(This blog was originally published in 2018 for the [intermix.io](#) blog.)

[Gradient Boosting](#)[Python](#)[Predictions](#)[Classification](#)[Follow](#)

Written by Tina Wenzel

66 Followers · 19 Following

No responses yet



What are your thoughts?

[Respond](#)

More from Tina Wenzel

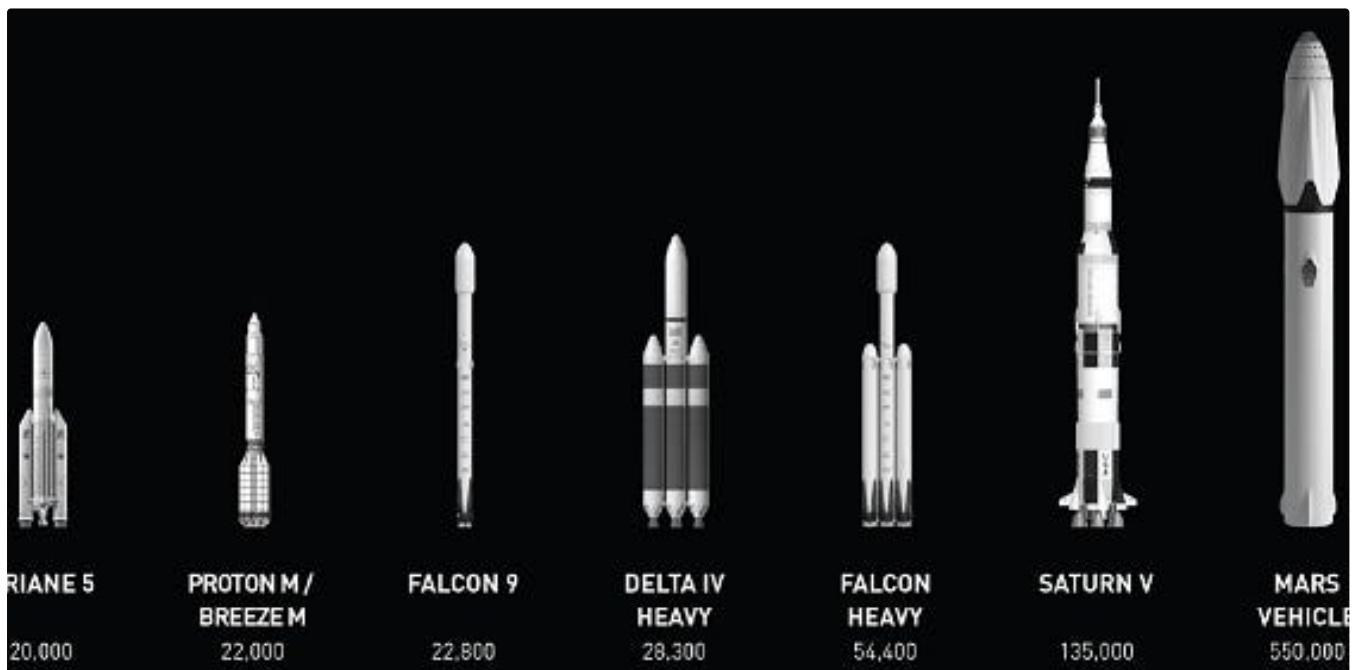


✳ In Carwow Product, Design & Engineering by Tina Wenzel

PostgreSQL vs. pandas — how to balance tasks between server and client side

By Tina Wenzel and Kavya Gupta.

Jul 19, 2017 🖱 1K 💬 5



 Tina Wenzel

Model consistency of Python gradient boosting libraries

The version of the library is not something that gets much attention usually. Few people care about whether they are using version 0.19.0...

★ Apr 12, 2018 🖱 76 💬 3

[See all from Tina Wenzel](#)

Recommended from Medium



20 Advanced Statistical

 Sarowar Jahan Saurav

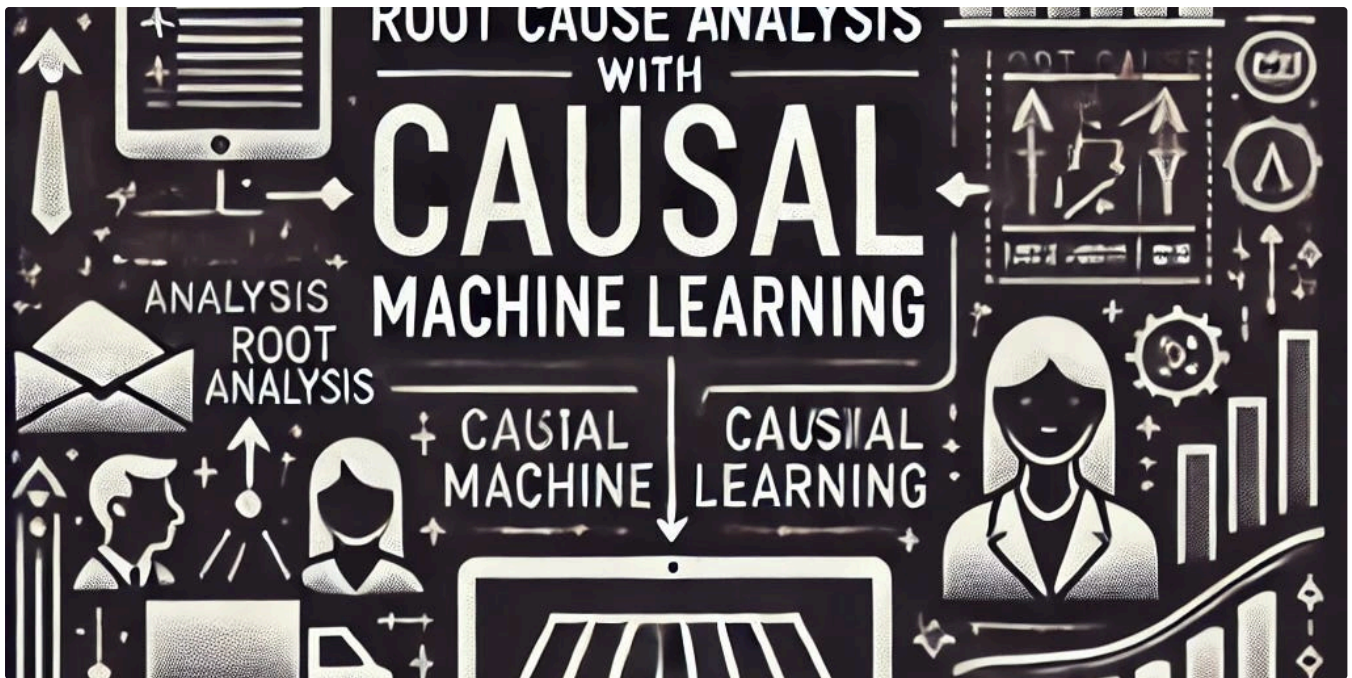
20 Advanced Statistical Approaches Every Data Scientist Should Know



Data science is a multidisciplinary field that combines mathematics, statistics, computer science, and domain expertise to extract...

Feb 6 🖱 1.2K 💬 19





 Ryuta Yoshimatsu

Causal ML for Root Cause Analysis

Introduction and Application of Causal Machine Learning in Root Cause Analysis

Feb 10  148  2

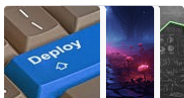


Lists



Coding & Development

11 stories · 1009 saves



Predictive Modeling w/ Python

20 stories · 1832 saves



Practical Guides to Machine Learning

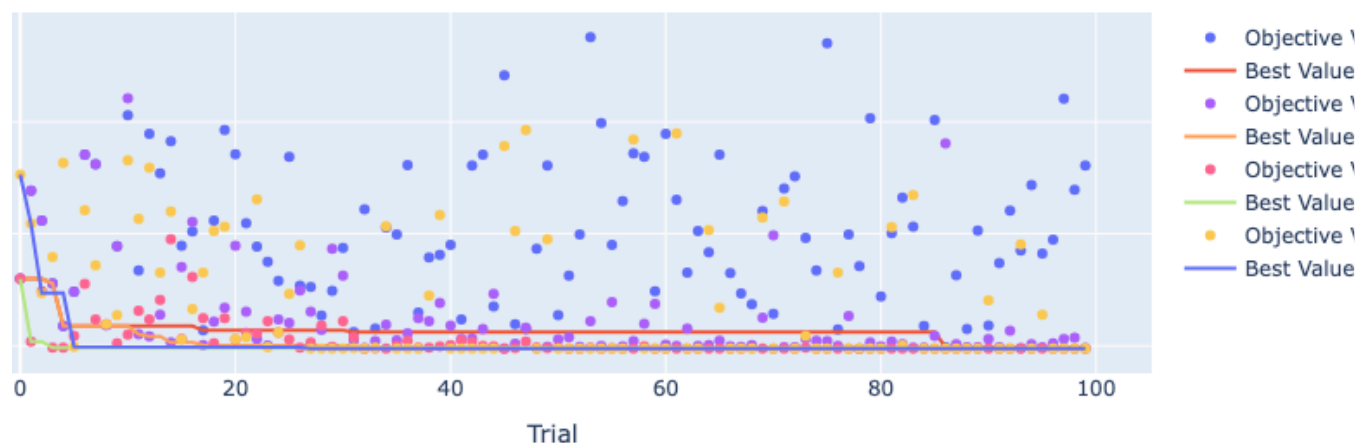
10 stories · 2208 saves



ChatGPT

21 stories · 971 saves

Optimization History Plot

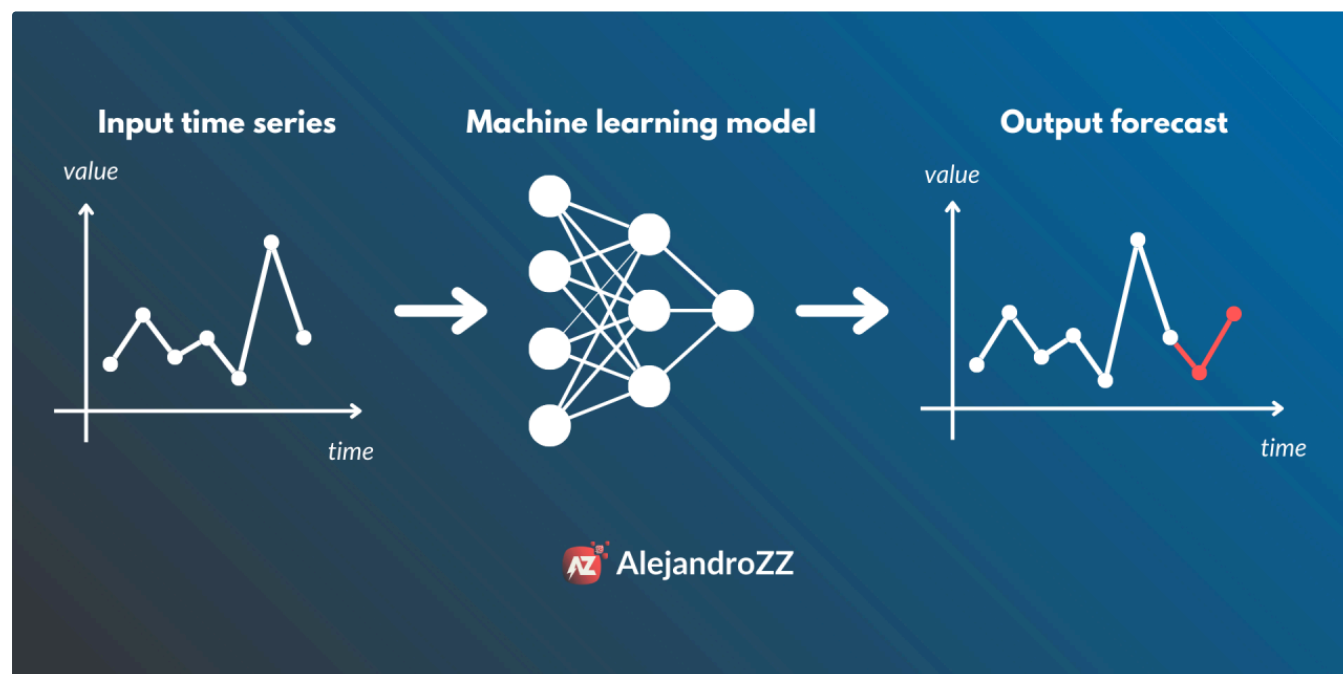


 In Optuna by c-bata

Announcing Optuna 4.2

We are pleased to announce the release of Optuna 4.2! Optuna 4.2 now supports several new optimization algorithms, a gRPC storage proxy for...

Jan 20  12



 Karan_bhutani

Feature Engineering for Time-Series Data: A Deep Yet Intuitive Guide.

How to Transform Raw Time-Series Data into Powerful Predictions

Feb 7 🖱️ 31 💬 2



yashwanth reddy

Forecasting Using Xgboost (lag and decomposition)

We all know how powerful Xgboost is as a machine learning model, for those who dont, you will find out in this blog. Before starting with...

5d ago 🖱️ 4



Document tabs +

Tab 1

Data Science Roadmap

For Your First Data Scienc...

Note: Most of the resourc...

Week 1: Basics of Python ...

Introduction to Python

Python Fundamentals

Operators and Control ...

Loops

Introduction to Strings

Week 2: Python Data Types

Week 3: Object-Oriented ...

OOP Part 2: Intermediat...

OOP Part 3: Advanced ...

Week 4: Advanced Data...

Data Science Roadmap

For Your First Data Science Job

Open-Source Data Science Resources
[Discord Community](#)

Note: Most of the resources are provided at the end of each week.

Week 1: Basics of Python Programming

Introduction to Python

- What is Python? A brief history and applications.
- Installing Python and setting up an IDE (e.g., VS Code, Jupyter Notebook).

Python Fundamentals

- Python Output:** `print()` function and basic syntax.
- Data Types:** Overview of key data types (int, float, string, list, etc.).
- Comments:** Single-line and multi-line comments.
- Variables:** Declaring and using variables, dynamic typing.



In Biased-Algorithms by Amit Yadav

Hidden Markov Models in Python

Aug 28, 2024



See more recommendations